Machine learning quantum materials

Today’s learning outcomes

- The relation between a many-body wavefunction and a machine learning problem
- Why some methods for solving wavefunctions can be used for machine learning
- Why methods from machine learning provide insights in phases of quantum materials

Plan for today

- Neural-networks for quantum many-body wavefunctions
- Tensor-networks for supervised learning
- Machine learning density functional theory
- Machine learning phase transitions
- Using machine learning for quantum materials

Machine learning with neural networks

Some examples of machine learning

Supervised learning



“Dog”

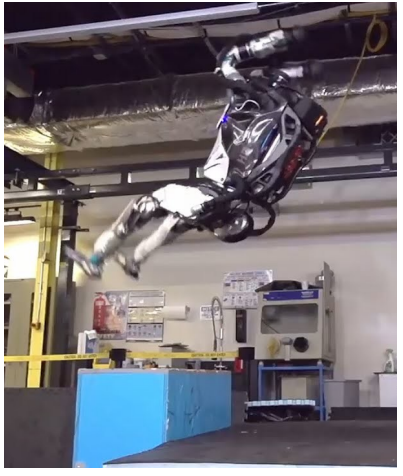
Labeled prediction

Generative learning



Probability Modeling

Reinforcement learning



Reward-based decision

(and others)

Generative machine learning: Realistic cat faces

Generative adversarial learning allows to create ultra-high quality realistic fake data





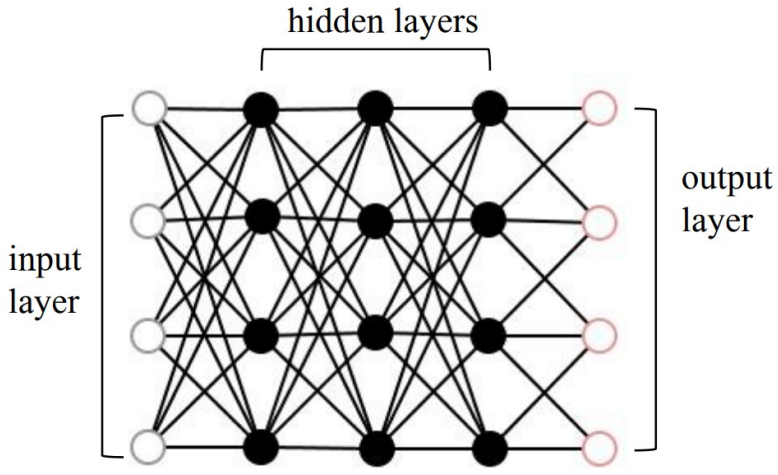


https://thesecatsdonotexist.com/

The basics of deep neural networks

Deep neural networks are “general function approximators”

$\vec{x}$



$\vec{y}$

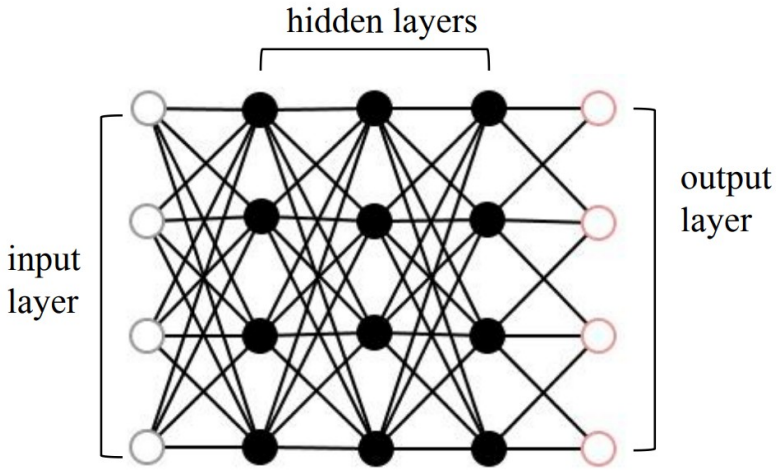
A deep neural network parametrices a function of the form $f(\lambda, \vec{x}) = \vec{y}$

λ Parameters of the function

$\vec{x}$ Input of the function

The basics of deep neural networks

Neural networks are a
family of parametric functions



The parameters are optimized to minimize a certain functional

$$\chi = \mathrm{LOSS}[\vec{y}_{\mathrm{real}} - \vec{y}_{\mathrm{predicted}}] = \mathrm{LOSS}[\vec{y}_{\mathrm{real}} - f(\vec{x}_{\mathrm{real}})]$$

For example $\chi \sim |\vec{y}_{\mathrm{real}} - f(\vec{x}_{\mathrm{real}})|^2$

A simple classification problem

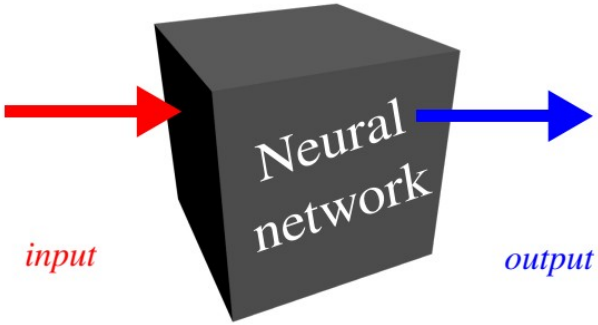




How to distinguish between two different animals with an algorithm?

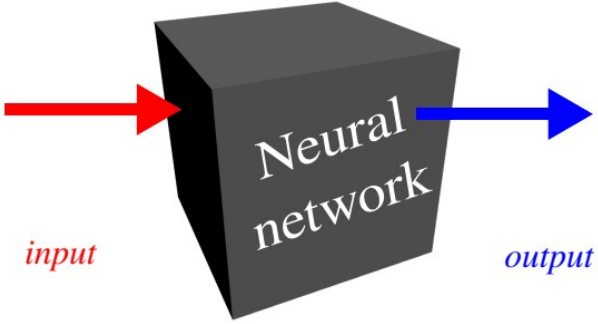
A simple classification problem





“Cat”





“Dog”

A simple classification problem

If we represent the image as a matrix, we just need to find the right function

$$\vec{x}_1$$



$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} & a_{06} & a_{07} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} & a_{17} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & a_{26} & a_{27} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & a_{36} & a_{37} \\ a_{40} & a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} & a_{47} \\ a_{50} & a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56} & a_{57} \\ a_{60} & a_{61} & a_{62} & a_{63} & a_{64} & a_{65} & a_{66} & a_{67} \\ a_{70} & a_{71} & a_{72} & a_{73} & a_{74} & a_{75} & a_{76} & a_{77} \\ a_{80} & a_{81} & a_{82} & a_{83} & a_{84} & a_{85} & a_{86} & a_{87} \\ a_{90} & a_{91} & a_{92} & a_{93} & a_{94} & a_{95} & a_{96} & a_{97} \end{bmatrix}$$

$$f(\vec{x}_1) = \vec{y}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

“Cat”

$$\vec{x}_2$$



$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} & a_{06} & a_{07} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} & a_{17} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & a_{26} & a_{27} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & a_{36} & a_{37} \\ a_{40} & a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} & a_{47} \\ a_{50} & a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56} & a_{57} \\ a_{60} & a_{61} & a_{62} & a_{63} & a_{64} & a_{65} & a_{66} & a_{67} \\ a_{70} & a_{71} & a_{72} & a_{73} & a_{74} & a_{75} & a_{76} & a_{77} \\ a_{80} & a_{81} & a_{82} & a_{83} & a_{84} & a_{85} & a_{86} & a_{87} \\ a_{90} & a_{91} & a_{92} & a_{93} & a_{94} & a_{95} & a_{96} & a_{97} \end{bmatrix}$$

$$f(\vec{x}_2) = \vec{y}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

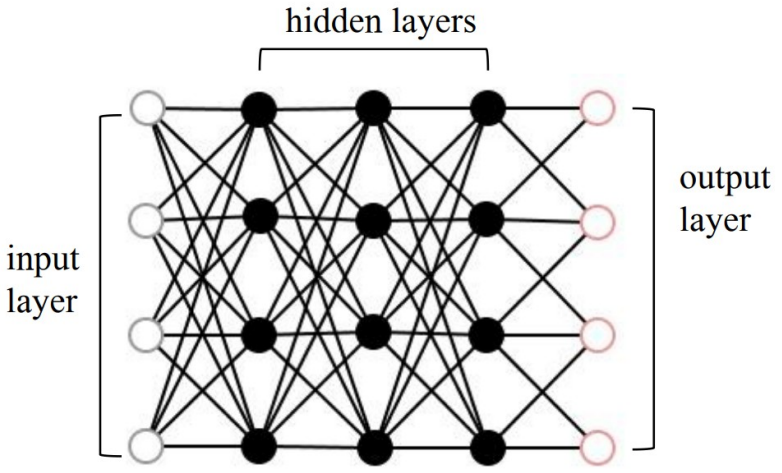
“Dog”

How do we find the function implementing this operation?

Supervised learning in a nutshell

Take a neural-network

Input the image (NxN matrix)
Output a 2D vector



Take a few examples and minimize $\chi = \mathrm{LOSS}[\vec{y}_{\mathrm{real}} - f(\vec{x}_{\mathrm{real}})]$

After the minimization (training), the neural-network will be able to classify new examples

Neural network quantum states

The quantum many-body problem

Let us go back to a simple many-body problem $\mathcal{H} = \sum_{ij} J_{ij} \vec{S}_i \cdot \vec{S}_j$

A typical wavefunction is written as

$$|\Psi\rangle = \sum c_{s_1, s_2, \ldots, s_L} |s_1, s_2, \ldots s_L\rangle$$

We need to determine in total 2^L coefficients

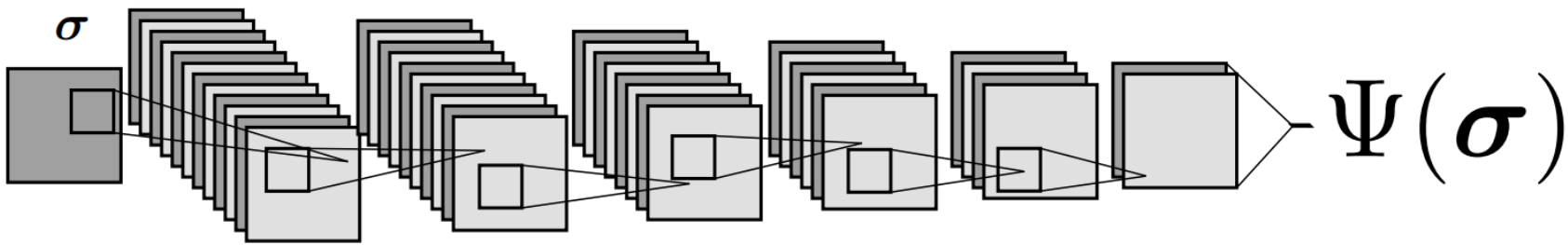
We can use a neural network to map the basis element to its coefficient

Neural-network quantum states

Do not store the coefficient, but find the right function that generates them

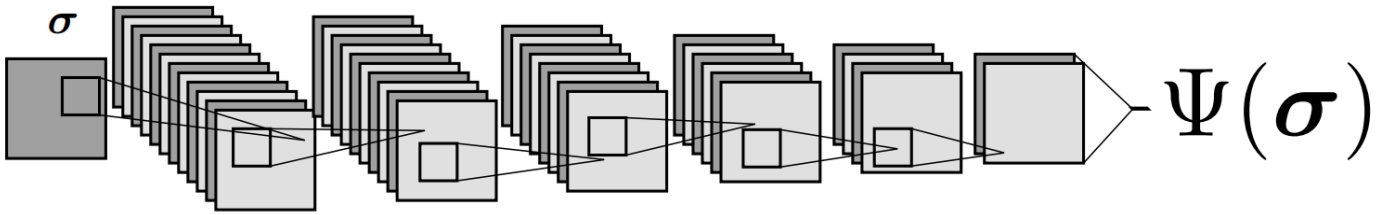
$$c_{s_1,s_2,\ldots,s_L} = f(s_1, s_2, \ldots, s_L)$$

Deep neural network



The idea is similar as tensor networks, but exploiting a machine-learning architecture

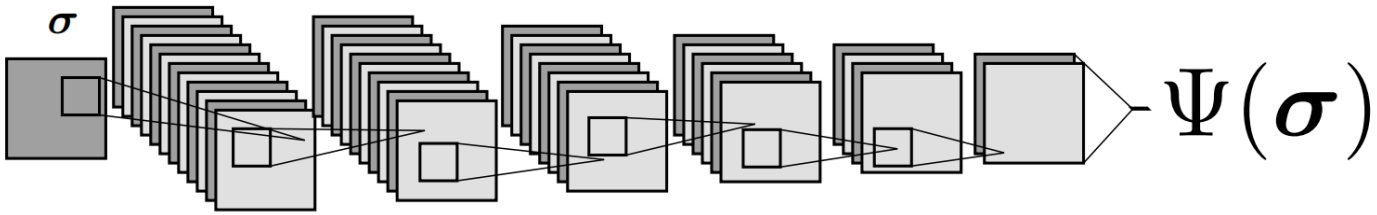
Neural-network quantum states



How do we find the right neural network for the ground state?

$$|\Psi\rangle = \sum c_{s_1,s_2,..,s_L} |s_1, s_2, ...s_L\rangle \qquad c_{s_1,s_2,....,s_L} = f(s_1, s_2, ..., s_L)$$

Neural-network quantum states



How do we find the right neural network for the ground state?

$$|\Psi\rangle = \sum c_{s_1,s_2,..,s_L} |s_1, s_2, ...s_L\rangle \qquad c_{s_1,s_2,....,s_L} = f(s_1, s_2, ..., s_L)$$

Optimize the parameters of the network to minimize $E = \dfrac{\langle\Psi|H|\Psi\rangle}{\langle\Psi|\Psi\rangle}$

Finding the ground state with neural network quantum states

The variational principle implies that we need to minimize the energy to find the ground state

$$E(\boldsymbol{\theta}) = \frac{\left\langle \Psi_{\boldsymbol{\theta}} | \hat{H} | \Psi_{\boldsymbol{\theta}} \right\rangle}{\langle \Psi_{\boldsymbol{\theta}} \mid \Psi_{\boldsymbol{\theta}} \rangle} \geq E_0$$

Iteratively minimize the energy until convergence is reached

Compute the energy and the gradient

$$E(\boldsymbol{\theta}) \qquad \frac{\partial E(\boldsymbol{\theta})}{\partial \theta_p}$$

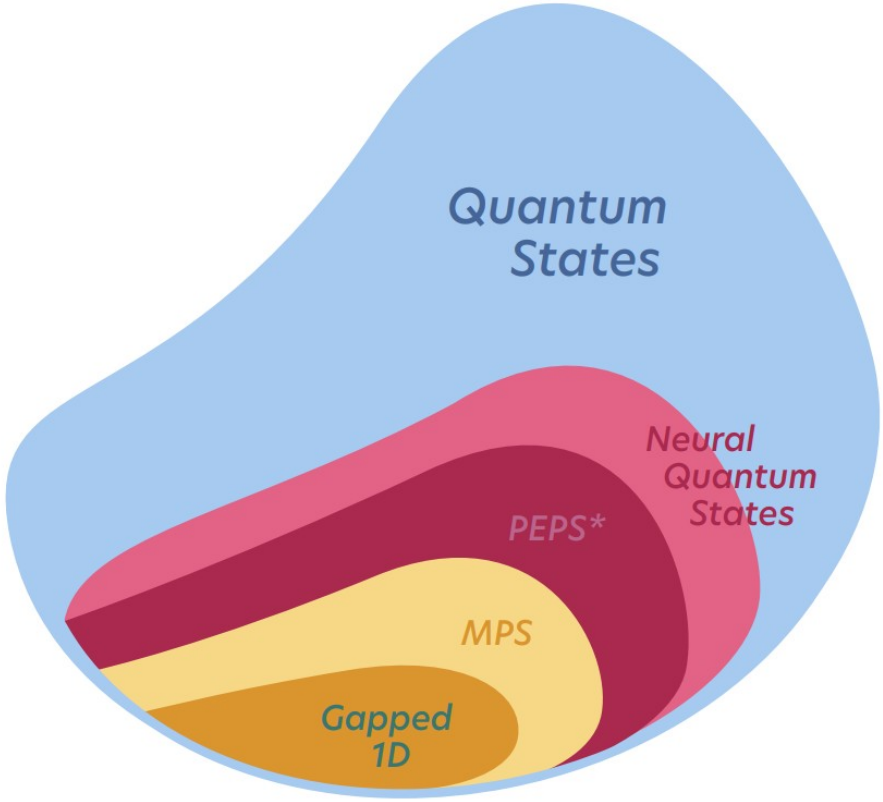
Redefine the variational parameters for the next iteration

$$\theta_p^{(s+1)} = \theta_p^{(s)} - \eta \frac{\partial E(\boldsymbol{\theta})}{\partial \theta_p}$$

Learning rate

The hierarchy of quantum states

Different variational states allow accessing different parts of the Hilbert space



MPS: matrix product states

PEPS: projected entangled pair states

Quantum state tomography

Imagine that you have an unknown many body state $|\Psi\rangle$



How do we reconstruct the wavefunction from the different projective measurements?

Quantum state tomography

A typical wavefunction is written as

$$|\Psi\rangle = \sum c_{s_1,s_2,\ldots,s_L} |s_1, s_2, \ldots s_L\rangle$$

We need to determine in total 2^L coefficients

The probability of obtaining each element in a projective measurement is

$$P(|s_1, s_2, \ldots, s_L\rangle) = |c_{s_1,s_2,\ldots,s_L}|^2$$

How can we determine the wavefunction without an exponentially large number of measurements?

Neural-network quantum state tomography

What is the most likely wavefunction consistent with a set of measurements?

Take a parametrization $\psi_{\lambda,\mu}(\mathbf{x}) = \sqrt{\frac{p_\lambda(\mathbf{x})}{Z_\lambda}} e^{i\phi_\mu(\mathbf{x})/2}$

Neural-networks

Enforce that the parametrization captures the measured data with maximum likelihood

$$\left|\psi_{\lambda,\mu}\left(\mathbf{x}^{[b]}\right)\right|^2 \simeq \left|\Psi\left(\mathbf{x}^{[b]}\right)\right|^2$$

Parametrized density

Measured densities

The many-body wavefunction as large data problem

The quantum many-body problem

Let us go back to a simple many-body problem $\mathcal{H} = \sum_{ij} J_{ij} \vec{S}_i \cdot \vec{S}_j$

A typical wavefunction is written as

$$|\Psi\rangle = \sum c_{s_1,s_2,\ldots,s_L} |s_1, s_2, \ldots s_L\rangle$$

We need to determine in total 2^L coefficients

Is there an efficient way of storing so many coefficients?

The matrix-product state ansatz

For this wavefunction $\left|\Psi\right\rangle = \sum c_{s_1,s_2,\ldots,s_L} \left|s_1, s_2, \ldots s_L\right\rangle$

Let us imagine to propose a parametrization in this form

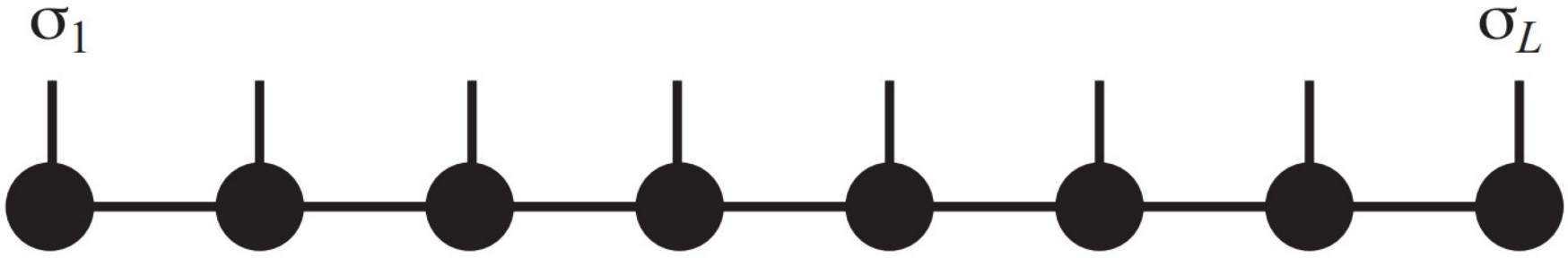
$$c_{s_1,s_2,\ldots,s_L} = M_1^{s_1} M_2^{s_2} \ldots . M_L^{s_3}$$

dimension 2^L

dimension $\sim LD^2$

(D dimension of the matrix)

The matrix product state representation

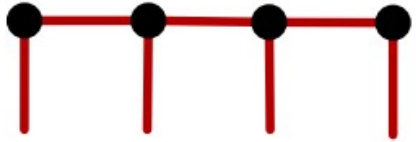


$$|\psi\rangle = \sum_{\boldsymbol{\sigma}} M^{\sigma_1} \dots M^{\sigma_L} |\boldsymbol{\sigma}\rangle$$

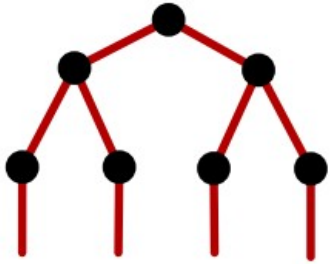
Generic tensor networks states

Tensor networks can be extended to deal with higher dimensional/critical systems

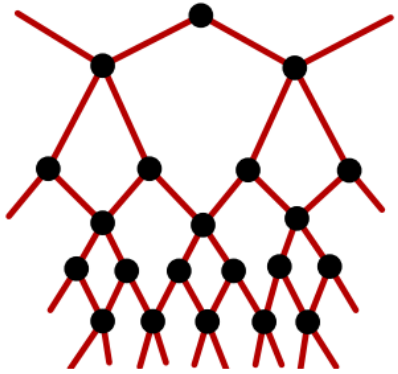
Matrix product state



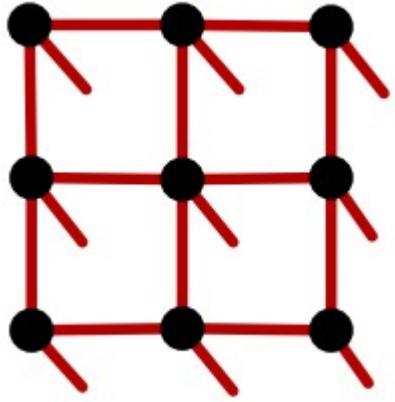
Tree tensor network



Multiscale renormalization ansatz



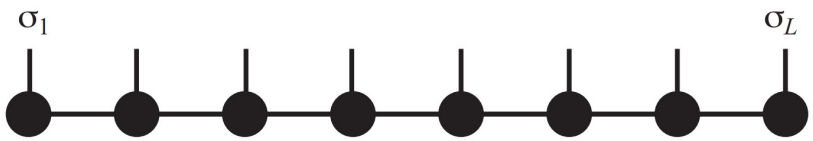
Projected-entangled pair states



Tensor network allow parametrizing high dimensional objects with different architectures

Using a tensor-network to classify images

Tensor-networks allow to parametrice high-dimensional functions

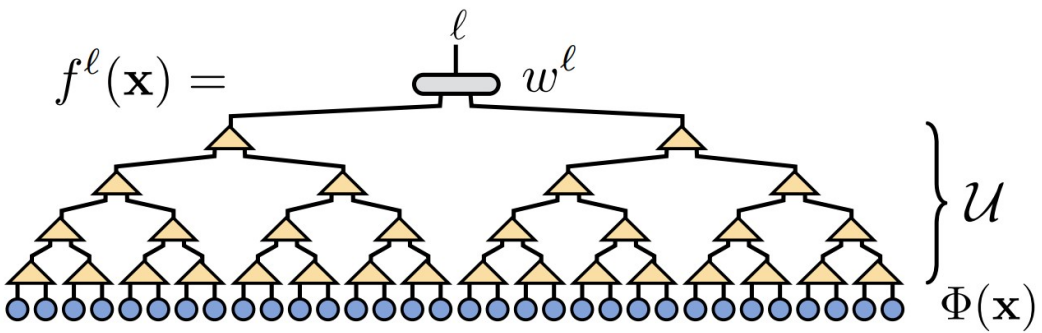


$$c_{s_1,s_2,....,s_L} = M_1^{s_1} M_2^{s_2} ....M_L^{s_3}$$

$$|\Psi\rangle = \sum c_{s_1,s_2,...,s_L} |s_1, s_2, ...s_L\rangle$$

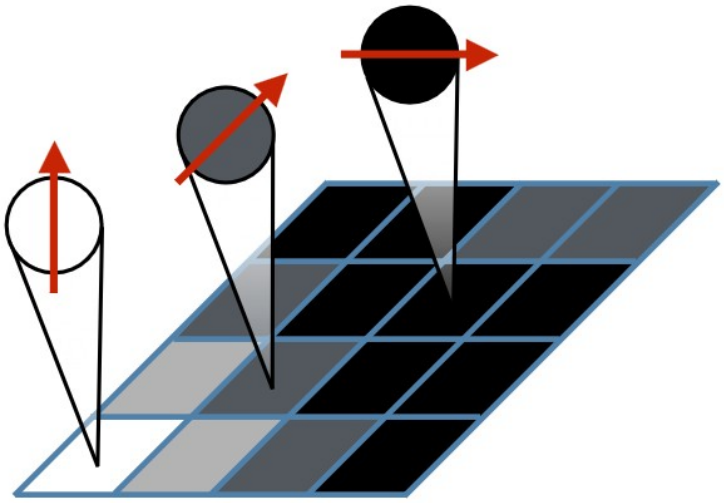
Can we use tensor-network architectures “as if” they were neural networks?

Quantum many-body inspired machine learning



Supervised machine learning with tensor networks

Map each pixel to a 2d vector



And perform the tensor-product over all indexes

$$\Phi^{s_1 s_2 \cdots s_N}(\mathbf{x}) = \phi^{s_1}(x_1) \otimes \phi^{s_2}(x_2) \otimes \cdots \phi^{s_N}(x_N)$$

And find a linear mapping that fulfills the classification task $f^{\ell}(\mathbf{x}) = W^{\ell} \cdot \Phi(\mathbf{x})$

Using a tensor-network to classify images

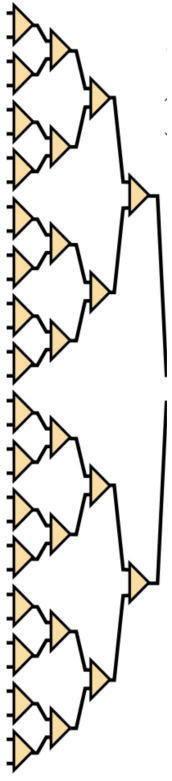


$\mathbf{x}$

$\Phi_1(\mathbf{x})$

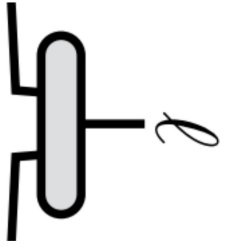


$\Phi_2(\mathbf{x})$



$\tilde{\Phi}_1(\mathbf{x})$

$\tilde{\Phi}_2(\mathbf{x})$



ℓ

$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$

$\begin{pmatrix} 0 \\ 1 \end{pmatrix}$

$$\tilde{\Phi}^{t_1 t_2}(\mathbf{x}_j) = \sum_{s_1, s_2, \ldots, s_N} \mathcal{U}^{t_1 t_2}_{s_1 s_2 \cdots s_N} \Phi^{s_1 s_2 \cdots s_N}(\mathbf{x}_j) \ .$$

The fashion MNIST dataset classified with tensor-networks



Results

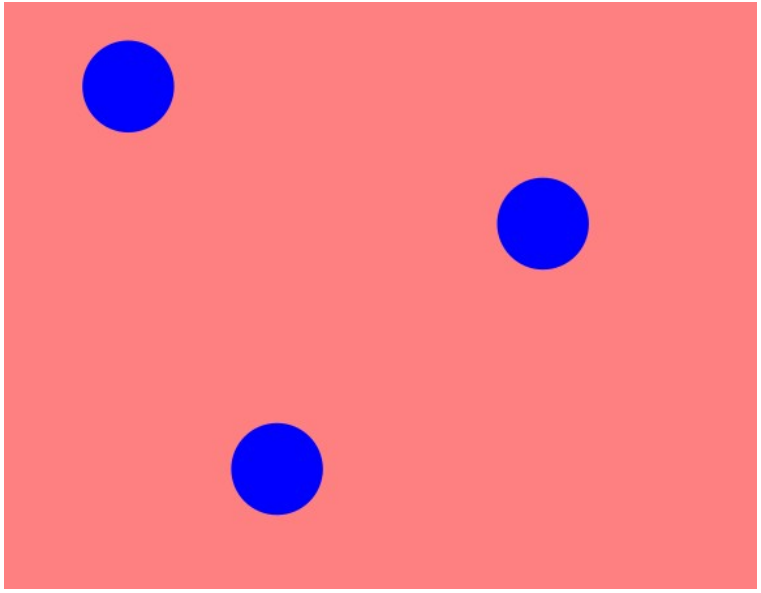
95.38% accuracy in training

88.97% accuracy in testing

Machine learning in density functional theory

The fundamental idea of density functional theory

Take the positions of all the atoms



And find the electronic density of the ground state (not necessarily the wavefunction)

The many-electron problem

The Hamiltonian for electrons in a solid

$$H_{\mathrm{el}} = -\frac{1}{2}\sum_{j=1}^{N}\nabla_j^2 - \sum_{j=1}^{N}\sum_{l=1}^{M}\frac{Z_l}{\tilde{r}_{jl}} + \sum_{j=1}^{N}\sum_{k>j}^{N}\frac{1}{r_{jk}}$$

Has an associated electronic density

$$\rho(\boldsymbol{r}) = N\int \mathrm{d}^3\boldsymbol{r}_2 \cdots \int \mathrm{d}^3\boldsymbol{r}_N \left|\Psi\left(\boldsymbol{r}, \boldsymbol{r}_2, \cdots, \boldsymbol{r}_N\right)\right|$$

The Hohenberg-Kohn theorem

For the ground state of a system, there is a one-to-one relation between the electronic density and the many-body wavefunction (Hohenberg-Kohn theorem)

$$\rho_0 \leftrightarrow |\Psi\rangle$$

The total energy can be written as a functional of the electronic-density

$$E(\rho_0)$$

The ground state energy can be computed if we know the functional (which we do not)

The Kohn-Sham equations

We do not know the “true” functional for density-functional theory

Let us take an “imaginary” non-interacting electron gas, with the same density as the real one

$$F_{\mathrm{HK}}[\rho] = T_{\mathrm{S}}[\rho] + J[\rho] + E_{\mathrm{XC}}[\rho]$$

where

$$T_{\mathrm{S}} = -\frac{1}{2}\sum_{j=1}^{N} \left\langle \psi_j \left| \boldsymbol{\nabla}^2 \right| \psi_j \right\rangle$$

Mean-field electronic interaction

And this is approximated (LDA, GGA, metaGGA, etc)

We do not know the exchange-correlation functional, could it be machine learned?

Machine learning the exchange-correlation functional

In the Kohn-Sham equations

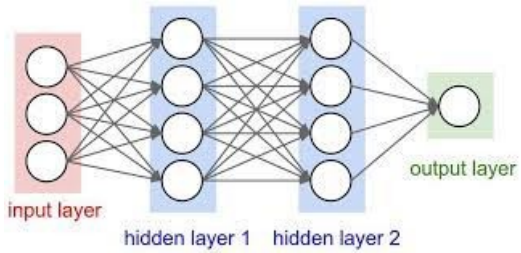
$$F[\rho] = T[\rho] + J[\rho] + E_{XC}[\rho]$$

The exchange-correlation functional is approximated (LDA, GGA, metaGGA, etc)

Instead of using a specific approximation, the functional can also be encoded as a neural-network

$$E_{XC} \equiv E_{XC}^{NN}$$

$$\frac{\delta E_{XC}}{\delta \rho} = V_{XC} \equiv V_{XC}^{NN}$$



We do not know the exchange-correlation functional, could it be machine learned?

Solving the Kohn-Sham equations

Given a certain density-functional $F_{KS}(\rho)$

An effective single-particle Hamiltonian must be solved, taking the form

$$H_{KS}|\psi_n\rangle = \epsilon_n|\psi_n\rangle$$

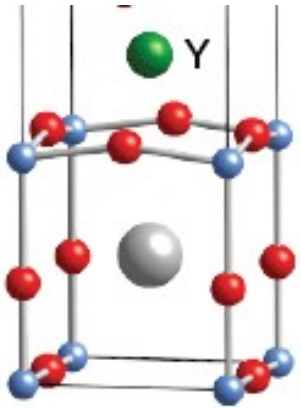
Where the Hamiltonian is obtained as a functional derivative

$$(H_{KS} - \epsilon_n)|\psi_n\rangle = \frac{\delta F_{KS}}{\delta|\psi_n\rangle} = 0$$

Solving the Kohn-Sham equations

Define your crystal
(atomic positions)

$V(\mathbf{r})$



Electronic density

$\rho(\mathbf{r})$

Hamiltonian

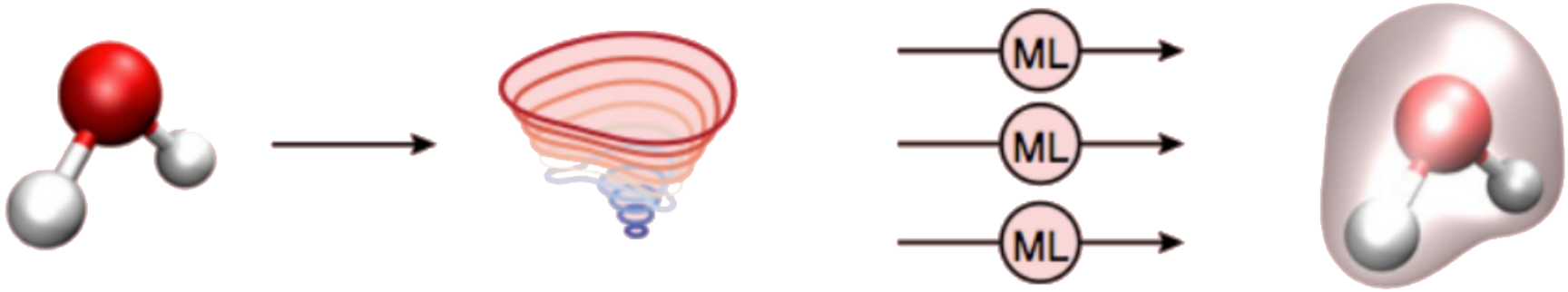
$H_{KS}(\rho)$

KS eigenvectors

$|\psi_n\rangle$

Problem #2: Solving the Kohn-Sham equations becomes too demanding for large systems

Machine learning the potential-density functional



Atomic positions

Electrostatic potential

Machine learning models

Electronic density

The neural network takes as input the potential, and as output the density

Phase classification with machine learning

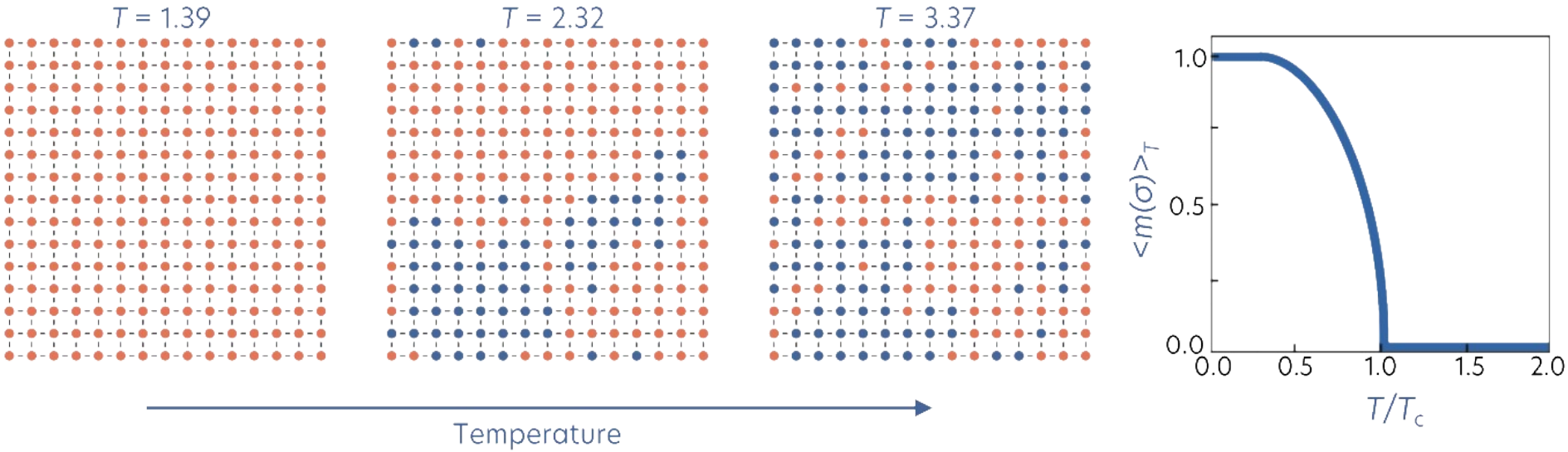
Combining machine learning with theory and experiments

- Phase classification in the Ising model (trivial) and Ising gauge theory (non-trivial)
- Extracting Hamiltonians from experimental observables

The Ising model

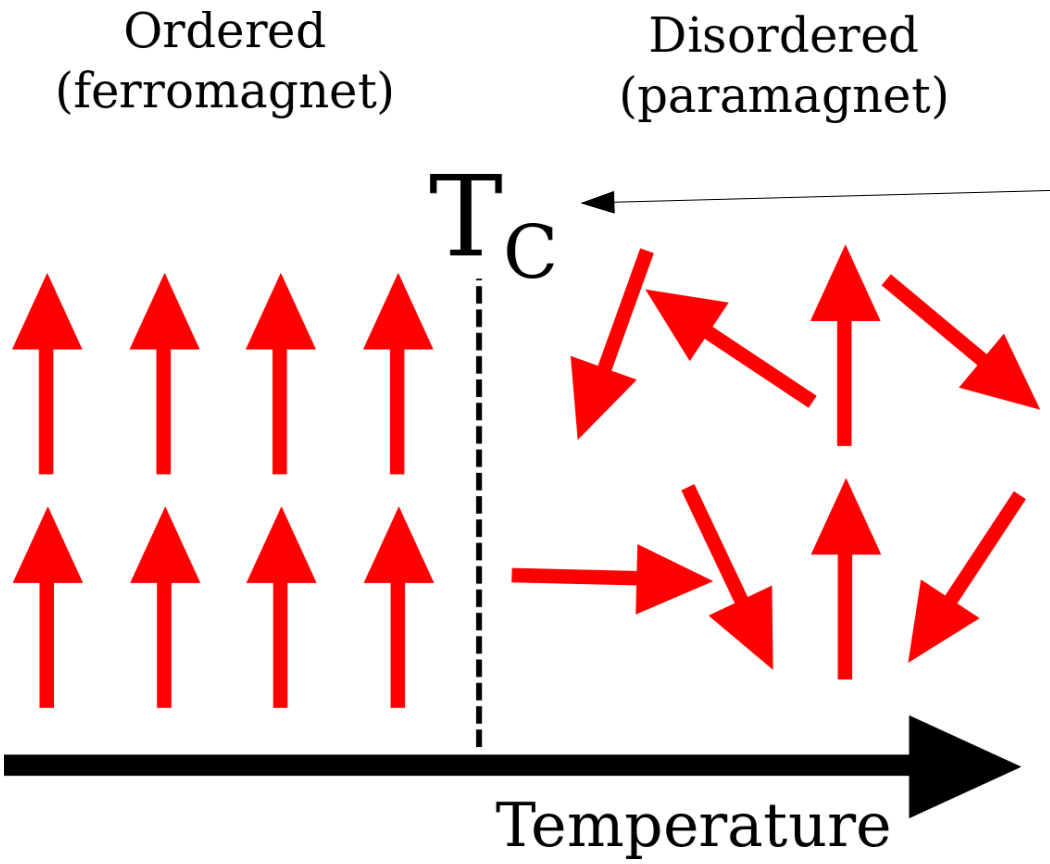
Take the Ising model and evaluate realizations at different temperatures

$$H(\sigma) = -J \sum_{\langle i,j \rangle} \sigma_i \sigma_j$$



How can we know in which phase we are from snapshots?

Classical phase transitions



Critical temperature

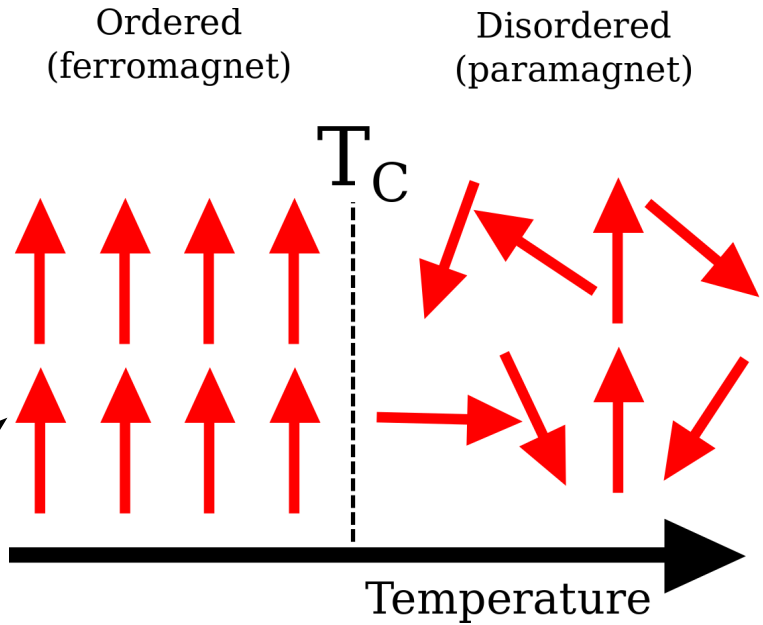
Can we use machine learning to find phase transitions?

2D Ising model

$$H = -J \sum_{\langle ij \rangle} s_i s_j$$

Classical phase transitions





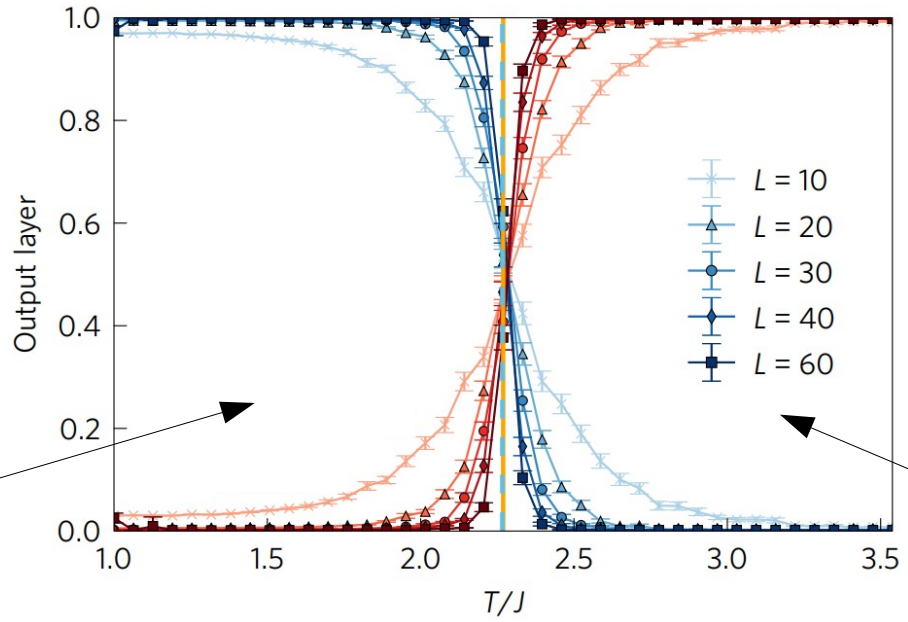
$$H = -J \sum_{\langle ij \rangle} s_i s_j$$

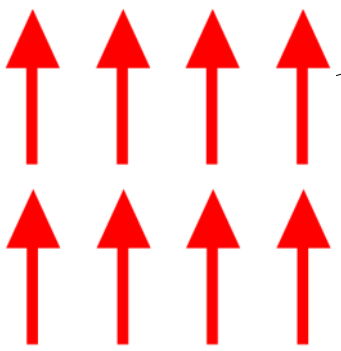


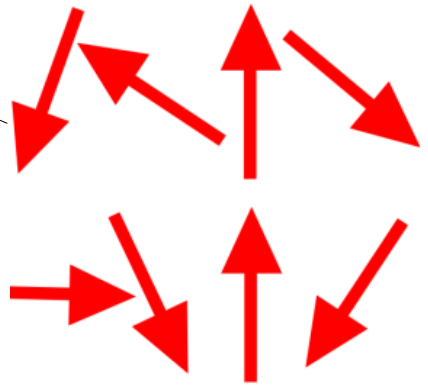
We can use a “classification algorithm” as did before

Classical phase transitions

$$H = -J \sum_{\langle ij \rangle} s_i s_j$$







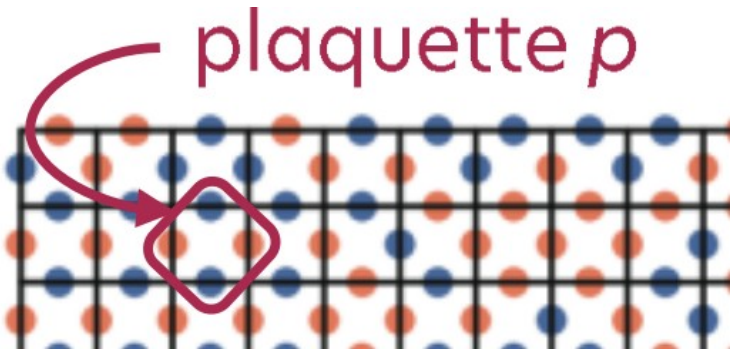
A neural-network learns to classify the two phases

A more challenging case: Topological phase transitions

Topological phase transitions do not have a local order parameter

Ising gauge theory $$H(\boldsymbol{\sigma}) = -J \sum_{p} \prod_{i \in p} \sigma_i$$

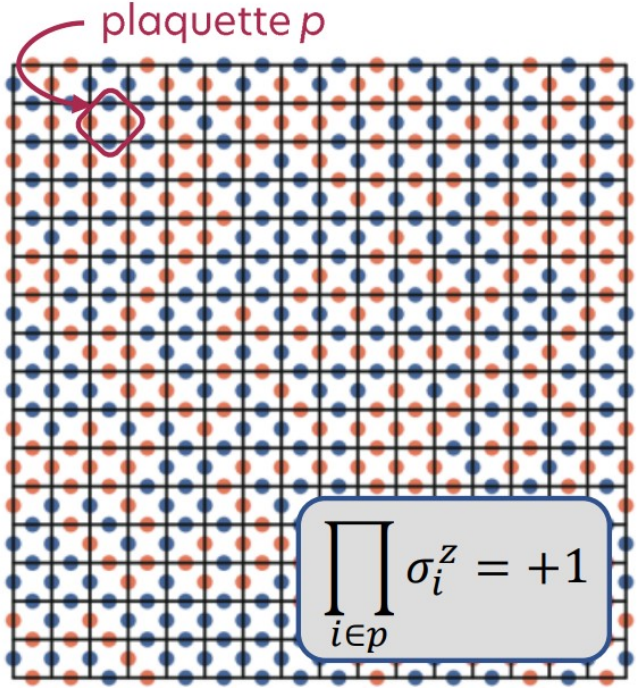
At low temperatures, the ground state fufills $$\prod_{i \in p} \sigma_i = 1$$



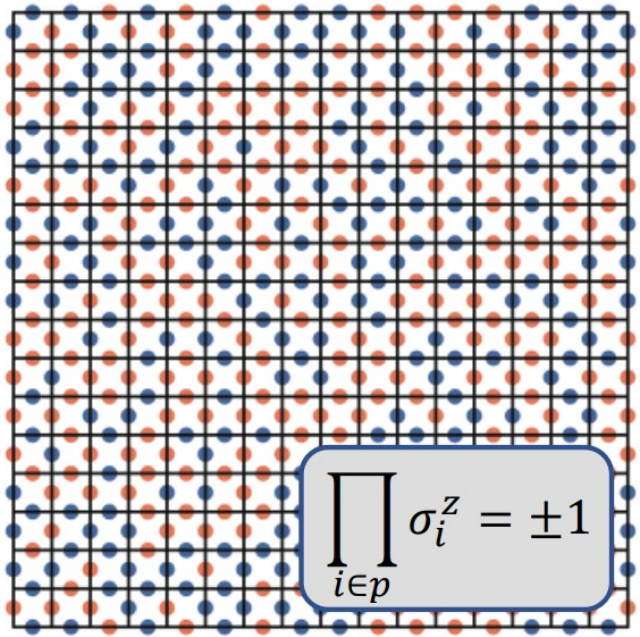
At high temperatures, the constraint is not fufilled

Topologically different phase in Ising gauge theory

$T \to 0$



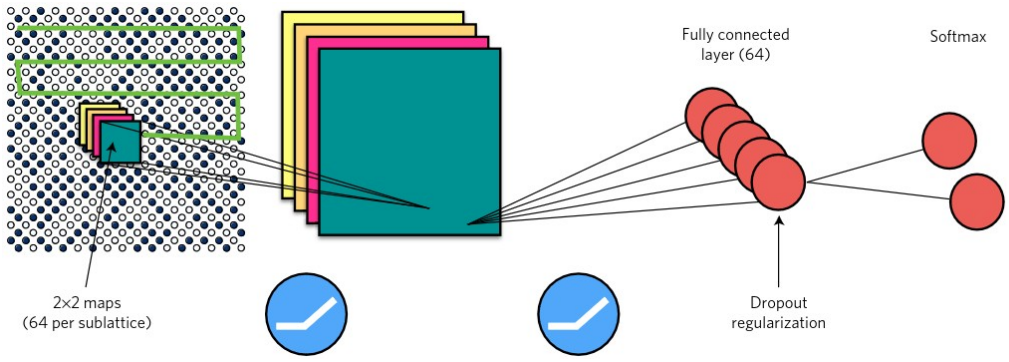
$T \to \infty$

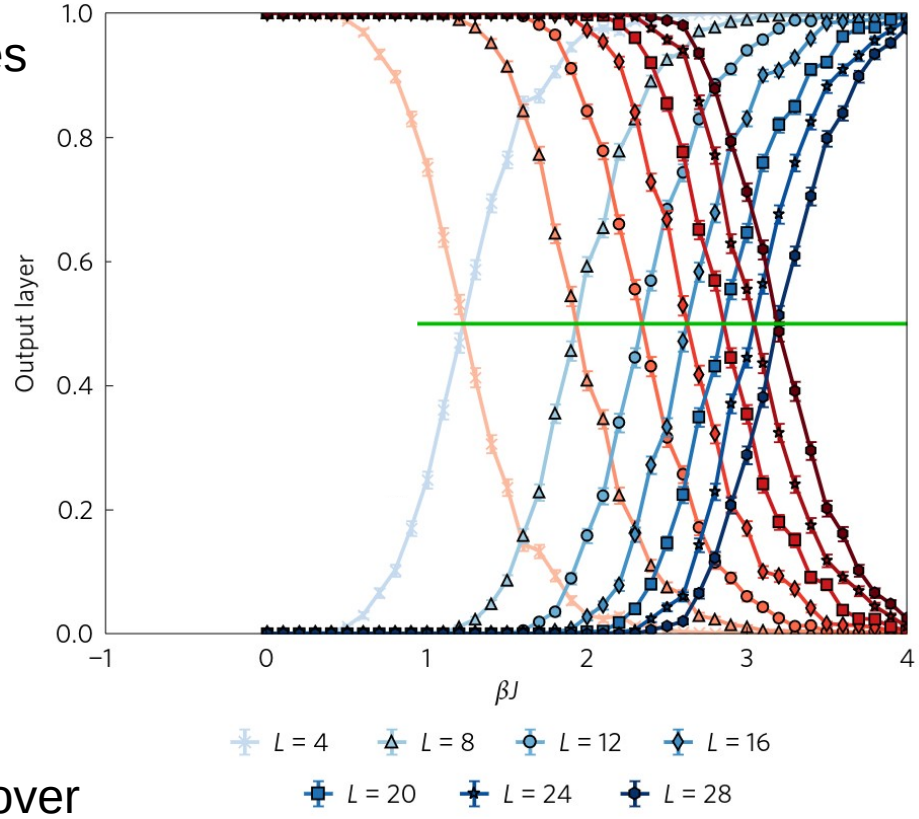


Low temperature and high temperature are topologically different, but the difference is not obvious

Topologically different phase in Ising gauge theory

Supervised learning can distinguish the two states





Capturing the size-dependent temperature crossover

Hamiltonian learning

The problem of Hamiltonian learning

How can we go from observables to the Hamiltonian?

Conventional workflow in theoretical physics

Solve the Hamiltonian

Compute an observable

$$H \longrightarrow \langle \Psi | O | \Psi \rangle$$

The problem we often find experimentally

Measure an observable

Determine its Hamiltonian

$$\langle \Psi | O | \Psi \rangle \longrightarrow H$$

How can we perform this task?

Navigating the manifold of Hamiltonians

How can we navigate a complex manifold of Hamiltonians

$$H(\lambda_i, w_i)$$

Parameters of our system
- Magnetic and electric fields
- Hopping
- Many-body interactions

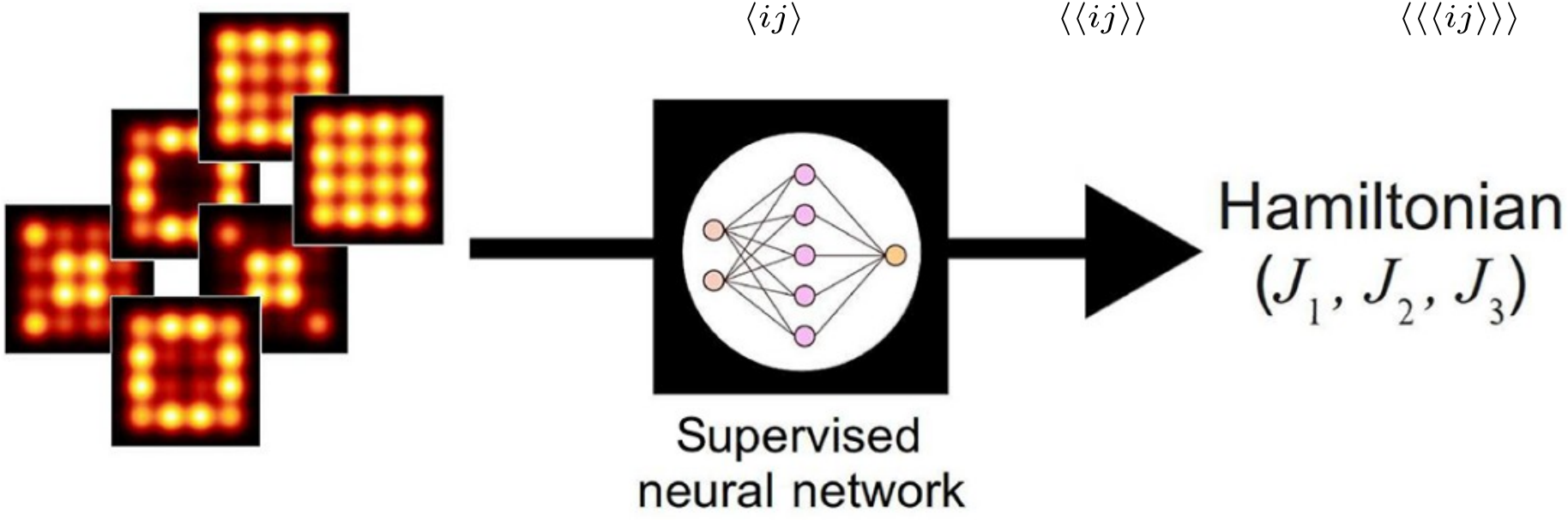
Hidden variables
- Smaller marginal parameters
- Disorder
- Computational limitations
- Pure randomness

How can we do it with high quality and limited amount of data?

The idea of Hamiltonian learning

Spectroscopy measurements

$$H = J_1 \sum_{\langle ij \rangle} \vec{S}_i \cdot \vec{S}_j + J_2 \sum_{\langle\langle ij \rangle\rangle} \vec{S}_i \cdot \vec{S}_j + J_3 \sum_{\langle\langle\langle ij \rangle\rangle\rangle} \vec{S}_i \cdot \vec{S}_j$$

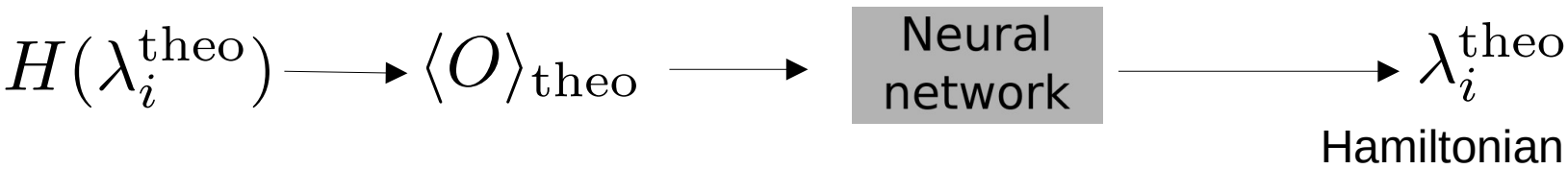


Provide the spectroscopy as input, obtain the Hamiltonian as output

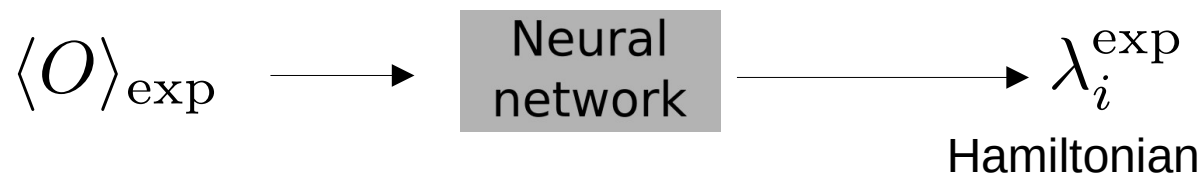
Hamiltonian learning: theory and experiment

Train a machine learning algorithm with simulated data

so that the parameters of each Hamiltonian are explicitly know



With the trained algorithm, evaluated in experimental data to extract the parameters



Take home

- Methods from quantum many-body physics can be used in machine learning
- Methods from machine learning can be used for quantum many-body physics
- Machine learning methods can be combined with conventional quantum materials methodologies